Student Portal with Claude Study Assistant

Project implementation report | Version 1.0 | 15 September 2026

Prepared by Sai Vikram Sake
Scope: completed local implementation through controlled private-note and shared-note access

Executive overview

This project combines authenticated study-note management and conversational learning in one FastAPI application. Students can create, edit and delete notes while using Claude Haiku 4.5 in an adjacent chat panel. PostgreSQL stores users, notes, conversations, messages and note-sharing permissions. Claude can retrieve a student's own notes through bounded tools and can also read notes that another student has explicitly shared.

The implementation now supports three sharing modes: private, available to all logged-in students, and available only to selected students. A recipient chooses the source student and up to five accessible notes before asking Claude a question. The backend verifies permission again before every request, and Claude identifies the note title and owner when using shared material.

Problem statement

A conventional notes portal stores information but does not let students investigate it conversationally. A standalone chatbot can explain a topic but cannot automatically see the student's saved material. Connecting a model directly to every database record would create a serious privacy risk, while copying notes manually interrupts study. Students also need a controlled way to share useful notes without giving others permission to edit or delete them.

Purpose and objectives

The objective is to provide one study workspace for note management, contextual assistance, persistent chat and permission-controlled collaboration. The system must authenticate every student, enforce ownership on the server, use deterministic code for exact operations such as word counting, and expose only approved note data to Claude. Shared access must be reversible and must clearly identify the student who supplied each note.

Implemented outcome

The local prototype demonstrates registration and login, session and JWT-cookie authentication, personal sticky notes, persistent conversation history, Markdown responses, cross-chat question retrieval, three private-note tools, note-sharing controls, recipient note selection and shared-note-aware Claude answers. Manual tests also confirm that making a shared note private prevents future retrieval.

Reading guide

The report covers architecture, database design, authentication, portal and chatbot features, tool calling, shared-note permissions, observed tests, operation and remaining production considerations. Screenshots appear beside the functionality they document.

System architecture and authentication

FastAPI provides the application server and validated routes. Jinja2 renders the portal pages, while HTML, CSS and JavaScript implement the dashboard, split view, history drawer and note selectors. SQLModel and SQLAlchemy connect the application to PostgreSQL. The Anthropic SDK sends permitted context and tool results to Claude Haiku 4.5.

Layer	Technology	Responsibility
Student interface	Jinja2, HTML, CSS and JavaScript	Login, sticky notes, chat controls, sharing dialogs and shared-note selection
Application server	Python 3.10, FastAPI and Uvicorn	Validation, authentication, authorization, API routing and tool execution
Persistence	PostgreSQL, SQLModel and SQLAlchemy	Users, notes, sharing grants, conversations and messages
AI service	Anthropic SDK and Claude Haiku 4.5	Natural-language answers and requests for declared tools
Safe reply rendering	Marked and DOMPurify	Markdown conversion followed by HTML sanitization

Authentication flow

Registration stores a password hash rather than readable credentials. Login supports a signed session cookie and an optional JWT-cookie path selected through Secure Login. The backend resolves the authenticated user and passes that server-controlled identity to protected routes. Logout clears both authentication mechanisms.

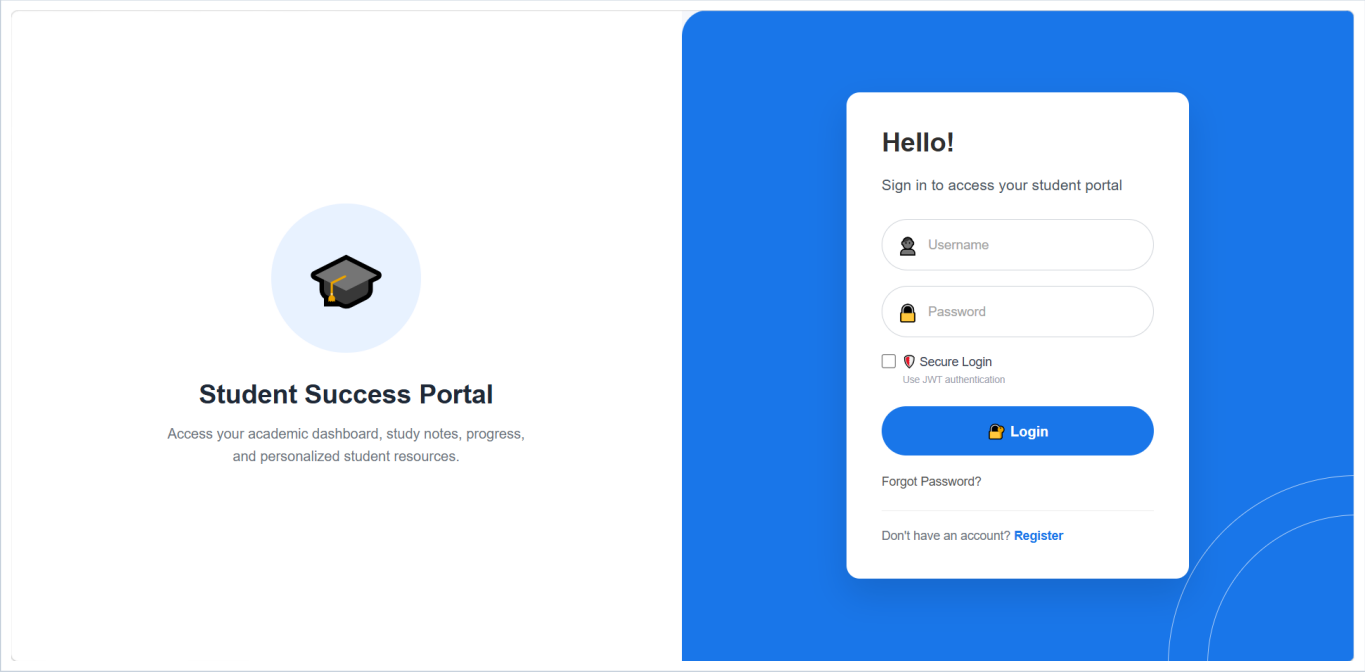


Figure 1. Student login page with standard and JWT-based Secure Login options.

Database design and access control

Ownership and sharing are represented explicitly in the relational model. A user owns notes and conversations; a conversation contains messages. A NoteShare row grants one recipient access to one note. Access is therefore decided by database state and authenticated identity rather than by model wording.

Table	Main fields	Relationship
public_user	id, username, password_hash	One user owns many notes and conversations
note	id, title, content, user_id, sharing_mode, created_at	user_id references public_user; sharing_mode controls visibility
note_share	note_id, recipient_user_id, created_at	Composite key grants selected-student access
conversations	id, user_id, title, created_at	Each conversation belongs to one user
messages	id, conversation_id, role, content, created_at	Each message belongs to one conversation

Authorization rules

Every note modification filters by both note ID and owner ID. Conversation listing is scoped to the current student, and reading, sending, renaming or deleting a conversation verifies ownership. Cross-chat retrieval joins messages to conversations and filters by the authenticated user. Private-note tools also obtain user_id from backend context; Claude cannot provide or replace it.

Shared-note access predicate

A different student's note is readable only when its sharing mode is all_students, or when its mode is selected_students and a matching NoteShare row exists for the current recipient. The owner retains edit and delete authority. Recipients receive read-only access for study-assistant use and cannot change sharing configuration.

Permission changes

Changing a note to private removes future shared retrieval. Updating selected recipients replaces the applicable grants. The chat route validates selected shared-note IDs before Claude is called, and the shared-note tool validates them again when content is requested. This defense-in-depth prevents stale interface selections from bypassing current database permission.

Security boundary

The model never receives a database password or an unrestricted SQL connection. Tool schemas expose only bounded operations. Database credentials and API keys remain in environment configuration, HTTP-only cookies reduce script access to login tokens, and inaccessible resource IDs return a non-disclosing not-found response.

Student dashboard and note management

After login, the dashboard shows the authenticated student's note count, a form for creating study notes, and sticky-note cards. Each owned card supports editing, deletion and sharing configuration. A compact symbol in the upper-right corner communicates its current access state: a lock for private, a globe for all students and a group symbol for selected students.

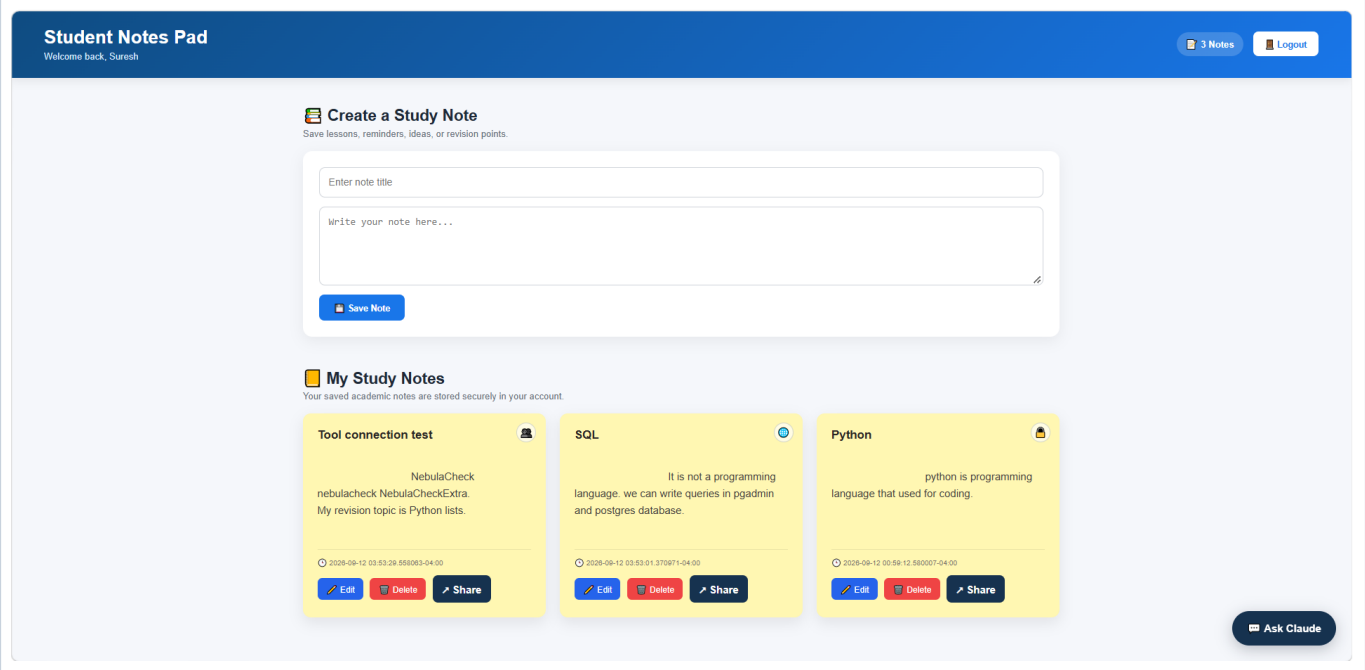


Figure 2. Notes dashboard showing owned notes, sharing-state symbols and Share controls.

Implemented note behavior

Capability	Behavior
Create	Stores a non-empty title and body under the authenticated student's user ID.
Edit	Loads and updates only a note owned by the current student.
Delete	Deletes only an owned note after user confirmation.
Share	Opens an owner-only configuration dialog for private, all-student or selected-student access.
Status	Displays a compact icon without occupying the note body; explanatory text remains available as a tooltip.

The dashboard remains the primary workspace even when Claude is available. A bottom-right Ask Claude control opens the assistant without navigating away from the student's notes.

Integrated study assistant experience

The assistant opens in a two-column workspace. Notes remain visible on the left, while the chatbot occupies the right. Each side scrolls independently, and closing the assistant returns the dashboard to full width. The iframe boundary keeps chatbot styling isolated while the authenticated cookies continue to identify the student to backend routes.

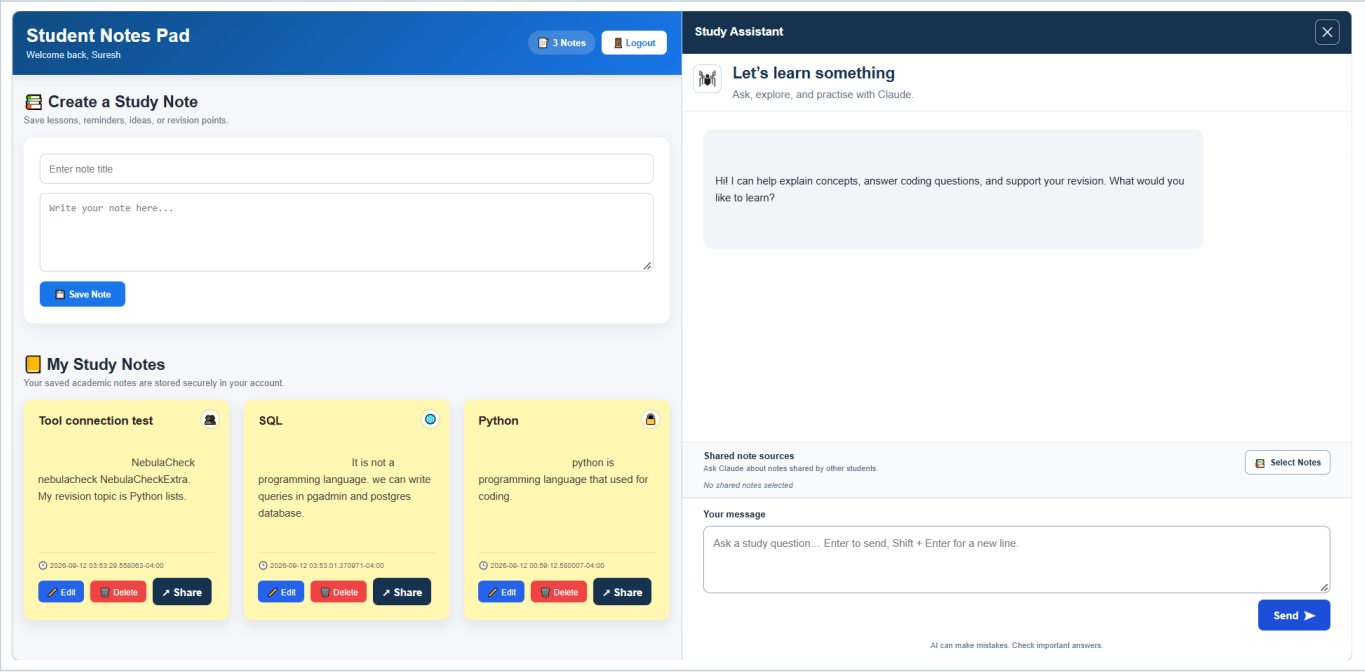


Figure 3. Split workspace with owned notes on the left and Claude on the right.

Chat capabilities

Capability	Implemented behavior
Message entry	Enter sends a question; Shift+Enter inserts a new line.
Readable replies	Markdown headings, lists, emphasis, tables and code blocks are rendered and sanitized.
Persistence	Messages and conversation metadata are stored in PostgreSQL.
Copy and export	Students can copy messages or code and download a conversation as text.
Conversation management	New chat, reopen, search, rename and delete operations are available.
Source selection	A Shared note sources control lets the student choose authorized external notes.

Conversation history and personal note tools

The image control beside the chatbot heading opens an overlay drawer. This keeps conversation management available without permanently shrinking the chat area. The drawer contains saved chats, title search, New Chat, download, rename and delete controls.

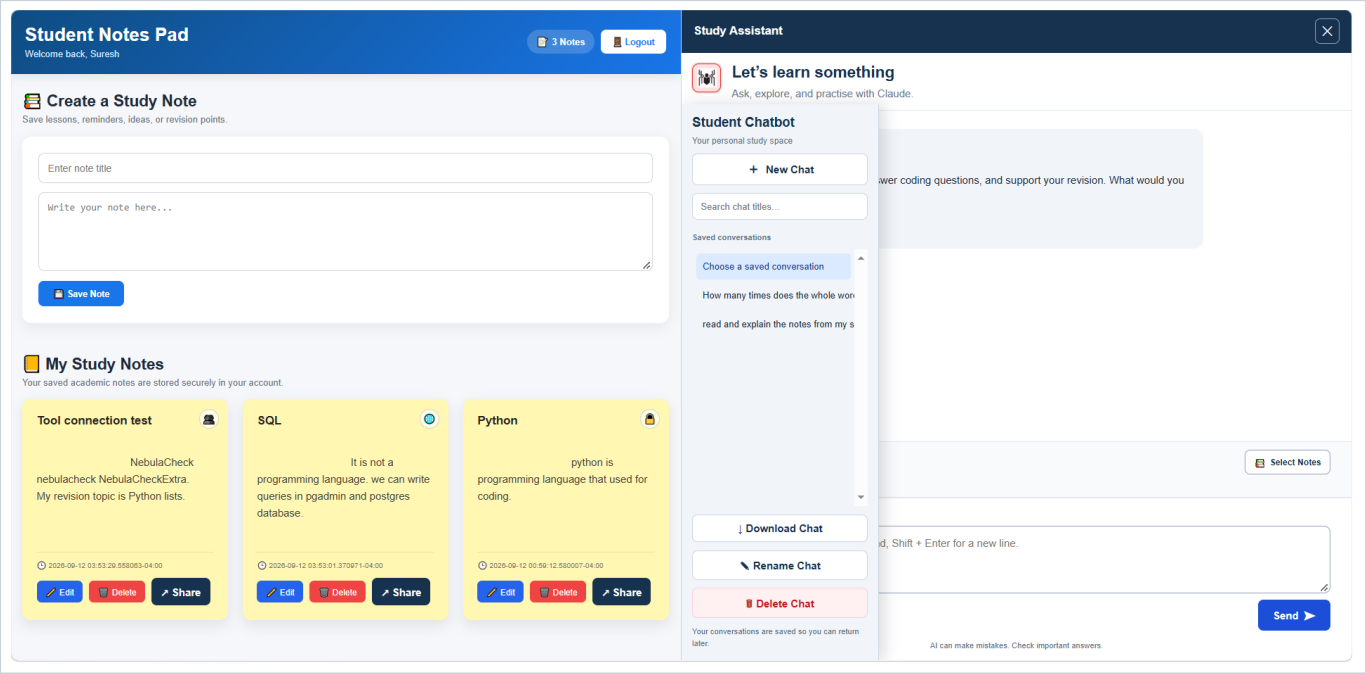


Figure 4. Chat history drawer with saved-conversation and management controls.

Persistent and cross-chat context

Within one conversation, the backend reloads saved messages before requesting the next answer. Across conversations, a memory service searches the same student's earlier user questions and supplies a small ranked set of matches. This retrieval is deliberately limited and never searches another student's history.

Tools for the student's own notes

Tool	Purpose and limits
count_word_in_notes	Counts case-insensitive whole-word occurrences across owned note titles and content using deterministic Python code.
search_notes	Searches owned titles and content for a literal phrase and returns limited previews.
read_note	Returns one owned note by ID, with a bounded content length and truncation indicator.

For the sample text 'NebulaCheck nebulacheck NebulaCheckExtra', the exact counter returns two whole-word matches. The longer word is correctly excluded. This illustrates why deterministic code is preferable to asking a language model to estimate an exact count.

Owner controlled note sharing

The note owner selects Share on an owned card and chooses who may read the note through the assistant. Private is the default. All students makes the note available to every other authenticated student. Selected students requires one or more recipient IDs and creates the corresponding NoteShare grants.

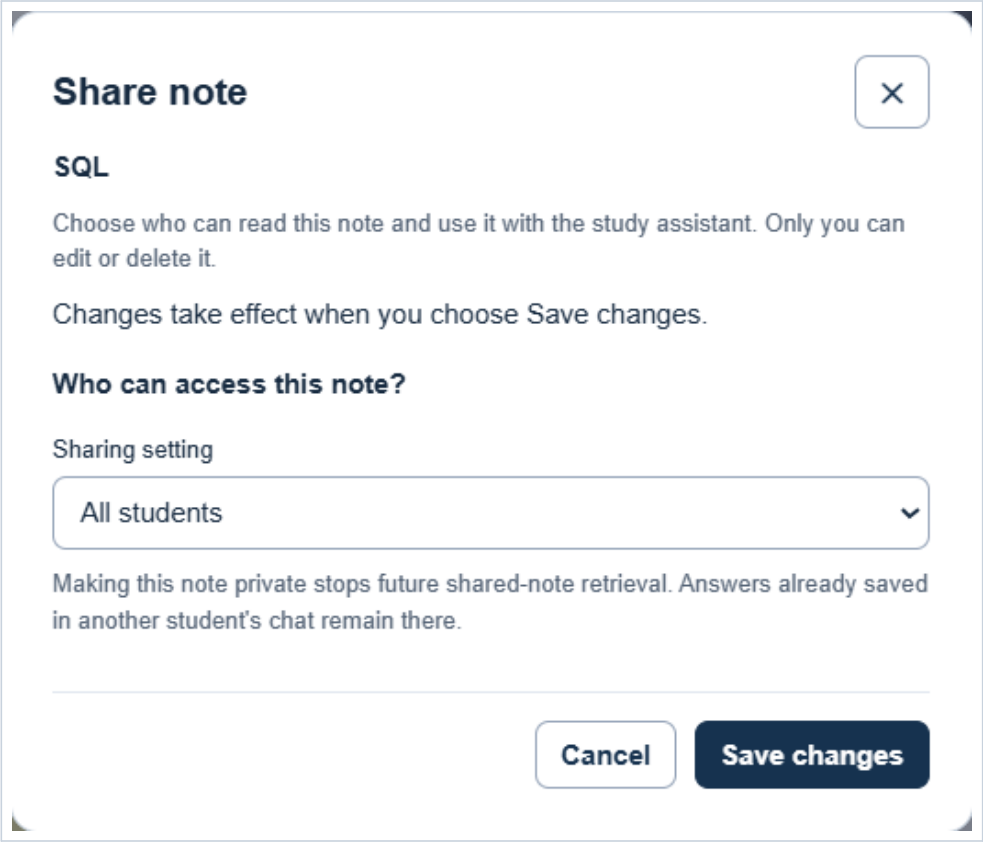


Figure 5. Owner sharing dialog configured to make the SQL note available to all students.

Student A and Student B example

Student A owns three notes: Python, SQL and Exam Plan. Python remains private, SQL is shared with all students, and Exam Plan is shared only with Student B. Student B can browse SQL and Exam Plan, select either as a Claude source, and see that Student A supplied it. Student C can browse SQL but cannot see Exam Plan. Neither recipient can edit or delete Student A's content.

Sharing mode	Who can retrieve the note	Stored state
Private	Owner only	sharing_mode is private; recipient grants are not used
All students	Every authenticated student except the owner through shared-note browsing	sharing_mode is all_students
Selected students	Only recipients chosen by the owner	sharing_mode plus matching NoteShare rows

Recipient selection and Claude retrieval

A recipient opens Select Notes inside the chatbot, chooses a source student and then selects up to five currently accessible notes. Selected items appear as removable chips containing the note title and owner. The browser sends only their note IDs with the chat request; it does not send note content directly.

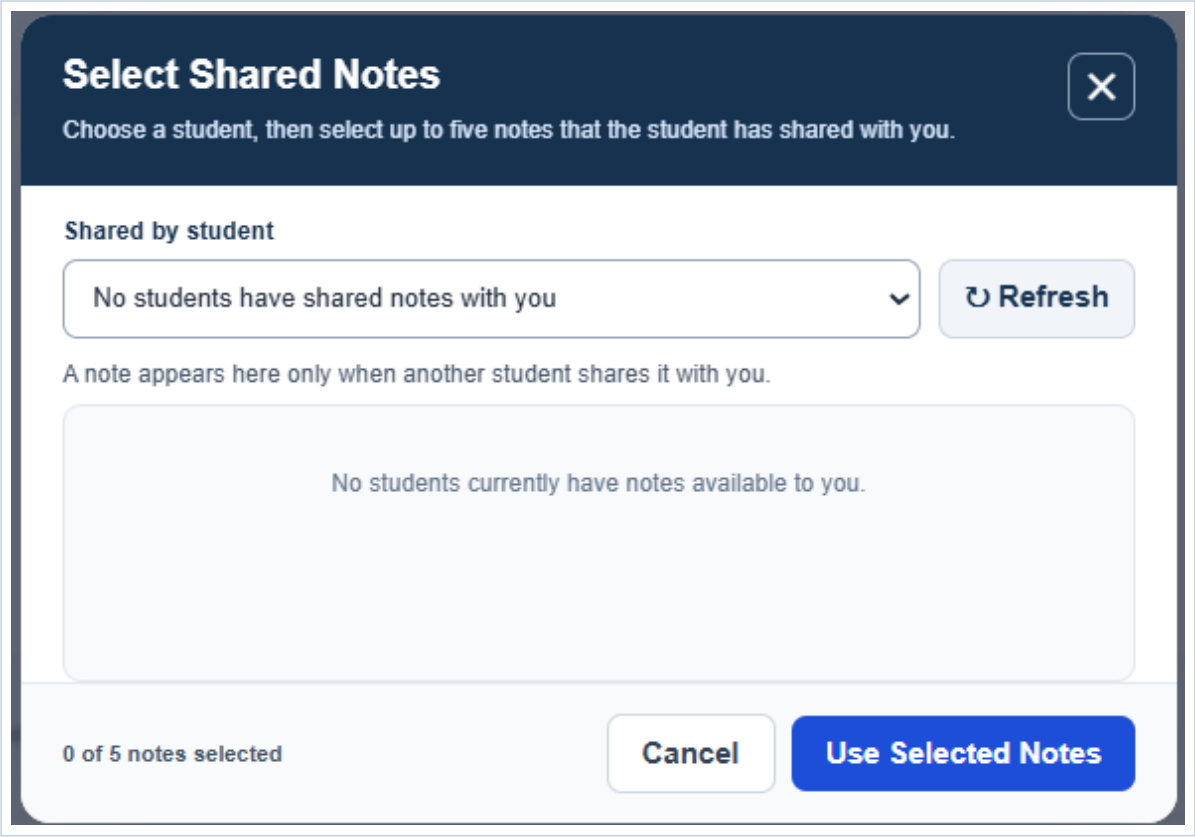


Figure 6. Recipient-side selector empty state before another student shares an accessible note.

Shared-note tool sequence

1. Browse permitted sources. Shared-note endpoints return only owners and note metadata satisfying the current access predicate. The interface never lists private or unauthorized selected-student notes.

2. Select bounded context. The recipient may choose at most five note IDs. Before sending, the interface rechecks each selection. Revoked entries are removed and the student is asked to review the remaining sources.

3. Validate on the server. POST /chat receives selected_shared_note_ids. FastAPI verifies all IDs for the authenticated recipient before invoking Claude. If any note is unavailable, the entire selected set is rejected rather than partially accepted silently.

4. Execute the declared tool. Claude may request read_selected_shared_notes. The executor ignores model-supplied ownership and uses the authenticated user plus the backend-maintained selection. Returned records include note title, owner username and bounded content.

5. Attribute the answer. System instructions require Claude to distinguish personal notes from selected shared notes and identify the source note and student. This gives Student B visibility into where the study material originated.

Testing and observed results

Validation used Swagger UI, the running portal and account switching between an owner and recipient. The results below record observed development behavior; they are not a substitute for an independent production security assessment.

Test	Observed result	Assessment
Authentication	Session and Secure Login paths opened the protected dashboard for the correct student.	Pass
Owned-note tools	Three notes were searched; NebulaCheck produced two whole-word occurrences and Note 5 was summarized.	Pass
Conversation ownership	Owner access succeeded; another student's conversation ID returned not found.	Pass
Cross-chat isolation	A student's earlier question could be retrieved; another account did not receive that record.	Pass
Sharing configuration	The owner changed Note 5 between private and selected_students and retrieved the saved recipients.	Pass
Recipient browsing	Student Test1 listed Suresh as a source and viewed Note 5 only while permission existed.	Pass
Shared Claude answer	Claude summarized Note 5 and identified Tool connection test, Suresh and Note ID 5.	Pass
Owner-only update	A recipient could not modify another student's sharing configuration.	Pass
Validation	selected_students with no recipients returned a clear 400 response.	Pass
Revocation	After the owner made Note 5 private, the recipient's chat request returned unavailable before Claude ran.	Pass

Interpretation

The tests demonstrate that shared retrieval follows current database permissions, that attribution reaches the final answer and that access revocation takes effect on future requests. The owner and recipient paths are distinct: sharing allows reading through approved endpoints and tools but does not transfer modification rights.

Accuracy and evaluation

Prompt evaluation remains useful for response-quality regression, while permission and counting tests require deterministic assertions. A model-generated explanation cannot independently prove that an authorization query was correct. For this reason, access checks occur in application code before model invocation and again inside the tool path.

Operation security considerations and outcome

Local setup and execution

The integration runs from the separate Student_portal_ai project folder and student_portal_ai PostgreSQL database so the original portal and chatbot remain untouched. Environment configuration supplies database settings, ANTHROPIC_API_KEY, ANTHROPIC_MODEL, SESSION_SECRET and JWT_SECRET. Secrets are excluded from this report and should also be excluded from Git source control.

With the virtual environment active and PostgreSQL running, start the development server using **python -m uvicorn main:app --reload --port 8001**. Open **http://127.0.0.1:8001/login** for the portal and **http://127.0.0.1:8001/docs** for API inspection.

Principal interfaces

Route or route group	Purpose
/login, /register and /logout	Authentication pages and actions
/dashboard and /assistant	Owned-note workspace and embedded chatbot
/add-note, /edit-note/{id}, /delete-note/{id}	Owner-controlled note creation, editing and deletion
GET or PUT /notes/{id}/sharing	Read or update sharing for an owned note
/sharing/students	Search possible recipients while configuring selected sharing
/shared-notes/students and /shared-notes	Browse source students and notes currently shared with the recipient
/shared-notes/{id}	Read one currently authorized shared note
/conversations and /conversations/{id}/messages	Manage owned conversations and retrieve messages
POST /chat	Generate and persist an answer using permitted personal or selected shared-note tools

Known limits

The system is a local development prototype. It does not yet demonstrate public deployment, HTTPS termination, production CSRF controls, rate limiting, centralized monitoring, backups, disaster recovery or concurrent-load behavior. Literal SQL retrieval is used rather than embeddings or vector similarity. Large shared libraries currently depend on paginated browsing rather than semantic ranking.

Project outcome

The completed implementation delivers an authenticated study portal with persistent chat, controlled access to personal notes and reversible student-to-student sharing. Claude receives note content only through server-executed tools after authorization. The final design therefore combines generative assistance with database-enforced ownership, explicit consent, source attribution and revocation.

The work is best described as tool-assisted retrieval with access-controlled generation. It follows retrieval-augmented principles, but the present retrieval mechanism uses relational queries and declared tools rather than a vector database. This approach is appropriate for exact ownership checks, word counts, note IDs and sharing relationships.